Rattrapage du TP1 au TP6

Démarche complète : commandes et résultats attendus

A. Larhlimi

Octobre 2026

Développement et déploiement d'applications intelligentes

Plan

Vue d'ensemble

La structure du projet

Ressources et préparation

TP1 : cadrer l'application et préparer l'environnement

TP2 : entraîner, évaluer et sauvegarder le modèle

TP3 : exposer le modèle par une API

Les tests de l'API

TP4 : tests approfondis et qualité

TP5 : l'interface Streamlit

TP6 : packager l'application

Vérification et notation par l'enseignant

Les messages d'erreur rencontrés

Annexes

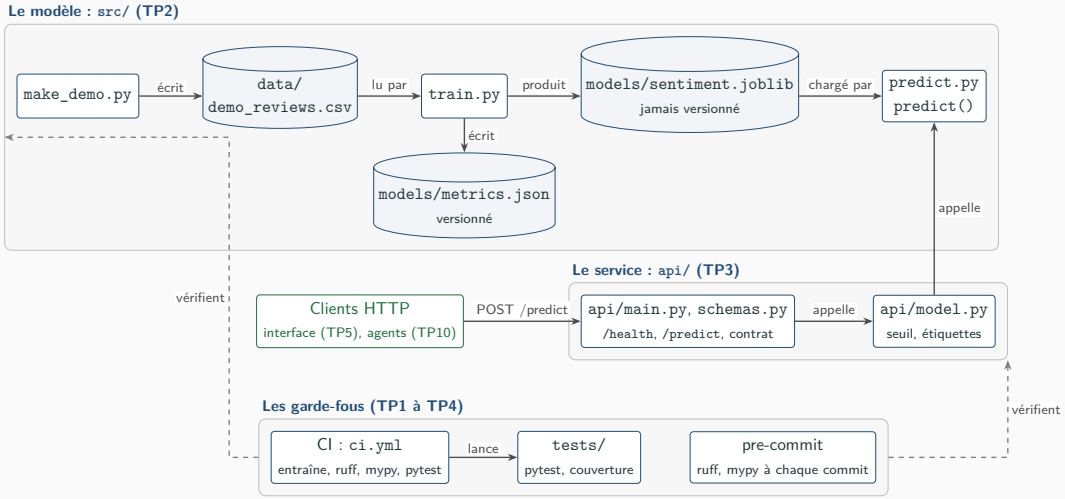
Vue d’ensemble

Le fil des six TP

TP	Sem.	Ce que le dépôt gagne	Contrôles	Tests à la fin
TP1	1	Environnement Python 3.11, structure du dépôt, trois fiches de cadrage, dépôt GitHub à deux auteurs	T0 à T7	38 passed
TP2	2	Jeu de démonstration, entraînement, évaluation, fonction `predict()`	C1 à C5	42 passed
TP3	3	API FastAPI (`/health`, `/predict`), contrat Pydantic, CI qui entraîne le modèle	T14 à T20	48 passed
TP4	4	Tests aux limites, abstention, intégration, ruff, mypy, pre-commit, CI étendue	T21 à T27	aucun `failed`
TP5	5	Interface Streamlit : un avis ou un fichier, verdicts, abstention, erreurs traduites	T28 à T34	aucun `failed`
TP6	6	Paquet installable : la roue, la commande `sentiment`, le modèle configurable	T35 à T41	aucun `failed`

Corrigé : 83, 95 puis 101 passed, et 2 xfailed, aux TP4, TP5 et TP6 ; un binôme peut en avoir plus, jamais avec un `failed`.

L'architecture du dépôt



Trois règles, et comment lire ce guide

Les trois règles d'architecture

1. L'API n'entraîne rien : elle appelle `predict()` du TP2, qui charge le modèle une seule fois.
2. Le modèle n'est jamais versionné : il se réentraîne en une commande ; seul `metrics.json` entre dans Git.
3. Les clients (interface du TP5, agents du TP10) passent par l'API, jamais par le fichier `.joblib`.

- Commandes pour PowerShell, dans `C:\IA-DEV\sentiment-app`, environnement activé.
- **Une commande par ligne** : quand une case contient deux commandes, lancez-les l'une après l'autre.
- Blocs verts : résultat attendu. Blocs orange : pièges. Un échec annoncé comme normal l'est vraiment.

La structure du projet

L'arborescence du dépôt en fin de TP5

```
sentiment-app/
  README.md                  présentation, installation
  TP1-ENONCE.md ... TP5-ENONCE.md
  MARCHE-A-SUIVRE*.md        le pas à pas de chaque TP
  TESTS.md AIDE-GIT.md DEPANNAGE.md
  requirements.txt           paquets de tout le semestre
  requirements.lock          versions exactes (TP1)
  requirements-dev.txt       outils de qualité (TP4)
  pyproject.toml             réglages de ruff et mypy (TP4)
  pytest.ini                 réglages de pytest
  .pre-commit-config.yaml    le crochet (TP4)
  .gitignore .env.example Makefile
  .github/workflows/ci.yml   intégration continue
  docs/
    cadrage/                 les trois fiches (TP1)
    semaine2.md ... semaine5.md
    journal-tp1-tp5.md       le journal de bord
    captures/                ses captures d'écran
  scripts/                   mise en place, diagnostic
  notebooks/                 la variante Colab (TP2)
```

```
data/
  sample_reviews.csv       10 avis d'exemple (TP1)
  demo_reviews.csv         290 avis, régénéré (hors Git)
  jeu_or.csv               20 avis du binôme (TP4)
  avis_exemple.csv         6 avis pour la page (TP5)
models/
  sentiment.joblib         le modèle (hors Git)
  metrics.json             sa mesure (dans Git)
src/                       le modèle (TP2, TP4)
  make_demo.py train.py evaluate.py predict.py
  courbe_seuil.py bilan_jeu_or.py
api/                       le service (TP3, TP4)
  __init__.py schemas.py model.py main.py
ui/                        l'interface (TP5)
  __init__.py client.py app.py
tests/                     97 tests (TP1 à TP5)
app/                       prévu au TP1, resté vide
.venv/                     l'environnement (hors Git)
```

Quatre couches, chacune à sa place

Dossier	TP	Ce qu'il contient	Ce qu'il ne fait jamais
`data/`	1 à 5	Les avis : exemples, jeu de démonstration, jeu d'or	Aucun calcul
`src/`	2, 4	Le modèle : générer, entraîner, évaluer, prédire, mesurer le seuil	Ne connaît ni HTTP ni l'interface
`models/`	2	Le modèle entraîné et sa mesure	Le modèle n'entre jamais dans Git
`api/`	3, 4	Le service : le contrat, l'abstention, les codes d'erreur	N'entraîne rien, ne connaît pas scikit-learn
`ui/`	5	La page et son client HTTP	N'importe jamais le modèle
`tests/`	1 à 5	Les vérifications automatiques de chaque couche	
`docs/`	1 à 5	Ce que le binôme écrit : fiches, comptes rendus, journal	
`scripts/`, `.github/`	1	Mise en place du poste, diagnostic, intégration continue	

Chaque couche n'appelle que celle du dessous : la page appelle l'API, l'API appelle `predict()`, `predict()` charge le modèle. Changer une couche ne touche pas les autres.

Ce qui est dans Git, et ce qui se reconstruit

Versionné : ce qui ne se recalcule pas

- Le code : `src/`, `api/`, `ui/`, `scripts/`
- Les tests : `tests/`
- Les écrits : `docs/`, les énoncés
- La configuration : `requirements*.txt`, `pyproject.toml`, `pytest.ini`, `.gitignore`, `ci.yml`
- Trois jeux d'avis écrits à la main : `sample_reviews.csv`, `jeu_or.csv`, `avis_exemple.csv`
- Une mesure : `models/metrics.json`

Hors de Git : ce qui se reconstruit

`.venv/`	`py -3.11 -m venv .venv`, puis `pip install`
`demo_reviews.csv`	`python src/make_demo.py`
`sentiment.joblib`	`python src/train.py`
`.env`	copie de `.env.example`
caches	recréés par pytest, ruff, mypy

Le `.gitignore` écarte `data/*` et `models/*`, sauf trois exceptions : `!models/metrics.json`, `!data/jeu_or.csv`, `!data/avis_exemple.csv`.

Le rôle de chaque fichier : la racine (1/2), les documents

Fichier	TP	Rôle
`README.md`	1	Présentation du dépôt, installation, tableau du binôme
`TPn-ENONCE.md`	1 à 5	L'énoncé de chaque TP : objectifs, déroulé, livrables, grille, pièges
`MARCHE-A-SUIVRE*.md`	1 à 5	Le pas à pas de chaque TP, contrôle par contrôle
`TESTS.md`	1	Les huit contrôles du TP1 et les trois états de référence
`AIDE-GIT.md`, `DEPANNAGE.md`	1	L'aide Git, et les blocages connus avec leur solution
`.github/workflows/ci.yml`	1 à 5	L'intégration continue : à chaque push, tout est vérifié sur une machine neuve

Le rôle de chaque fichier : la racine (2/2), la configuration

Fichier	TP	Rôle
`requirements.txt`	1	Les paquets de tout le semestre, versions figées
`requirements.lock`	1	Les versions exactes installées sur le poste du membre A
`requirements-dev.txt`	4	Les outils de qualité : pytest-cov, hypothesis, ruff, mypy, pre-commit
`pyproject.toml`	4	Les règles de ruff et de mypy
`pytest.ini`	1, 3	Les réglages de pytest ; `pythonpath = .` permet d'importer `api` et `ui`
`.pre-commit-config.yaml`	4	Le crochet : ruff et mypy avant chaque commit
`.gitignore`	1 à 5	Ce que Git ignore, et ses exceptions
`.env.example`	1	Le modèle du fichier `.env`, qui reste hors de Git
`Makefile`	1	Des raccourcis de commandes, pour Linux et macOS

Le rôle de chaque fichier : le modèle, src/ et models/

Fichier	TP	Rôle
`src/make_demo.py`	2	Écrit `data/demo_reviews.csv` : 290 avis, dont 100 mitigés et 30 neutres
`src/train.py`	2	Découpe, vectorise (TF-IDF), entraîne une régression logistique ; écrit le modèle et sa mesure. `--allocine` entraîne sur 20 000 vrais avis
`src/evaluate.py`	2	La matrice de confusion et les erreurs, à lire
`src/predict.py`	2, 4	`predict(texte)` : charge le modèle une seule fois, renvoie l'étiquette et la confiance. Typée au TP4
`src/courbe_seuil.py`	4	Pour chaque seuil : avis traités, exactitude, erreurs, avis vers un humain
`src/bilan_jeu_or.py`	4	Classe chaque avis du jeu d'or : juste, abstention, erreur franche
`models/sentiment.joblib`	2	Le modèle entraîné : hors de Git, reconstruit par `train.py`
`models/metrics.json`	2	Sa mesure (F1, source, taille du test) : versionnée, lue par les tests

Le rôle de chaque fichier : le service et l'interface

Fichier	TP	Rôle
`api/__init__.py`	3	Fait de `api/` un paquet : `from api.main import app`
`api/schemas.py`	3, 4	Le contrat : ce qu'une requête doit contenir (1 à 2000 caractères, pas que des blancs), ce qu'une réponse peut contenir (`Label`)
`api/model.py`	3, 4	Le raccord avec `predict()` : le seuil d'abstention, la correspondance des étiquettes, la vérification des classes au démarrage
`api/main.py`	3, 4	L'application FastAPI : démarrage (`lifespan`), `/health`, `/predict` ; 422, 503, 500
`ui/__init__.py`	5	Fait de `ui/` un paquet : `from ui import client`
`ui/client.py`	5	Tout ce que la page sait de l'API ; chaque réponse devient une `Reponse`
`ui/app.py`	5	La page Streamlit : un avis, ou un fichier d'avis
`data/jeu_or.csv`	4	Les 20 avis du binôme, avec leurs difficultés et leurs erreurs déclarées
`data/avis_exemple.csv`	5	Six avis pour essayer l'onglet « Un fichier d'avis »

Le rôle de chaque fichier : les 97 tests

Fichier	TP	Tests	Ce qu'il vérifie
`test_environment.py`	1	23	Python 3.11, environnement actif, paquets, structure, `.gitignore`
`test_cadrage.py`	1	12	Les trois fiches : rubriques, une approche, deux critères
`test_git_history.py`	1	3	Deux auteurs, `.venv/` jamais commité
`test_model.py`	2	4	Les métriques au-dessus du seuil, `predict()`
`test_api.py`	3	6	Les réponses de l'API
`test_limites.py`	4	10	Huit cas aux frontières du contrat, deux propriétés hypothesis
`test_integration.py`	4	6	Démarrage, modèle absent, 503, pannes en 500, étiquettes inconnues
`test_jeu_or.py`	4	21	Le jeu d'or complet, puis un test par avis
`test_client.py`	5	7	Le client de l'API
`test_interface.py`	5	5	La page, avec `AppTest`

Soit 97 tests : 96 passed et 1 xfailed dans le dépôt de l'essai, l'erreur déclarée du jeu d'or.

Le rôle de chaque fichier : docs/, scripts/, et les dossiers restants

Fichier ou dossier	TP	Rôle
`docs/cadrage/cas1` à `cas3`, `TEMPLATE.md`, `EXEMPLE-corrige.md`, `BANQUE-DE-CAS.md`	1	Les trois fiches à remplir, leur modèle, un exemple corrigé, douze cas de rechange
`docs/semaine2.md` à `semaine5.md`	2 à 5	Les comptes rendus : les observations et les réponses du binôme
`docs/journal-tp1-tp5.md`, `docs/captures/`	1 à 5	Le journal de bord, contrôle par contrôle, et ses captures
`scripts/setup.ps1`, `setup.sh`	1	Mise en place du poste : environnement, installation, `requirements.lock`, diagnostic
`scripts/check_setup.py`	1	Le diagnostic : une ligne par vérification, OK, ATTENTION ou ERREUR
`scripts/new_cadrage.py`, `init_depot.sh`, `verif_clone.sh`	1	Nouvelle fiche, création du dépôt, test du clone frais
`data/sample_reviews.csv`	1	Dix avis d'exemple
`notebooks/`	2	La variante Colab du TP2
`app/`	1	Prévu au TP1, resté vide : l'API vit dans `api/`

Qui appelle qui

Fichier	Appelle	Comment
`ui/app.py`	`ui/client.py`	`from ui import client`, après avoir ajouté la racine au chemin de Python
`ui/client.py`	l'API	Par HTTP (`httpx`), à l'adresse `API_URL` : jamais par un import
`api/main.py`	`api/model.py`, `api/schemas.py`	Imports du paquet `api`
`api/model.py`	`src/predict.py`	Ajoute `src/` au chemin, puis `from predict import ...`
`src/predict.py`	`models/sentiment.joblib`	`joblib.load`, une seule fois par processus (`lru_cache`)
`src/train.py`	`data/demo_reviews.csv`	Écrit le modèle et `metrics.json`
`tests/*.py`	`api.main`, `ui.client`	`TestClient` fait tourner l'API en mémoire ; `pythonpath = .`

Pourquoi cette chaîne compte

Elle ne remonte jamais : `src/` ignore l'API, l'API ignore la page. Un modèle réentraîné ne change que `models/` ; une API déplacée ne change que `API_URL`. C'est ce que les trois règles d'architecture protègent, et ce que les tests vérifient.

Ressources et préparation

Les ressources à télécharger

Ressource	TP	Pour qui	Contenu
Kit du TP1 : dépôt du cours, « Code » puis « Download ZIP »	1	binôme	Structure, 38 tests, fiches à remplir, scripts `setup` et `check_setup`, CI
`tp2-semaine2-kit.zip`	2	binôme	`src/`, `tests/test_model.py`, notebook Colab, énoncé, marche
`kit-tp3.zip`	3	binôme	`api/`, `tests/test_api.py`, `docs/semaine3.md` ; adapte `pytest.ini` et la CI
`tp4-tests-qualite-kit.zip`	4	binôme	Outils, courbe du seuil, bilan du jeu d'or, tests à compléter, `ci-tp4.yml`
`tp5-interface-kit.zip`	5	binôme	`ui/client.py`, `ui/app.py`, tests du client et de la page, avis d'exemple
`tp6-packaging-kit.zip`	6	binôme	Le paquet `sentiment_app`, six tests, `CHANGELOG.md`, extraits
`journal-tp1-tp6.md`	1 à 6	binôme	Le journal de bord : 53 captures, annexes A à E
`tp4-paquet.zip`, `tp6-paquet.zip`	4, 6	enseignant	Kit, corrigé, valeurs mesurées. **Ne pas distribuer.**
`releve_binomes.py`, `binomes.csv`	tous	enseignant	La relève automatique des dépôts

Dépôt du cours : `https://github.com/larhlimi2023/sentiment-app`. Les lots `ressources-etudiants.zip` et `ressources-enseignant.zip` regroupent ces fichiers.

Décompresser sans piège

L'Explorateur ajoute un dossier (`kit-tp3\kit-tp3`) : décompressez avec PowerShell, et donnez toujours aux scripts le chemin complet du dépôt.

```
New-Item -ItemType Directory -Force C:\IA-DEV
Expand-Archive "$HOME\Downloads\tp2-semaine2-kit.zip" -DestinationPath C:\IA-DEV -Force
Get-ChildItem C:\IA-DEV
```

Attendu

Un dossier par kit, directement dans `C:\IA-DEV`, à côté de `sentiment-app`. Jamais dedans.

Archive introuvable

Chemin inexistant : l'archive est ailleurs, ou renommée `(1)`. La chercher : `Get-ChildItem $HOME -Recurse -Filter "tp4*.zip"`.

Préparer le poste : Python 3.11, Git, VS Code

```
py -0
py -3.11 --version
git --version
```

Attendu

Une version 3.11 dans la liste, puis `Python 3.11.x`. Sinon : `winget install Python.Python.3.11`.

Pourquoi 3.11

Les tests du TP1 vérifient la version. Surtout, pandas 2.2 et scikit-learn 1.5 n'ont pas de version précompilée pour les Python plus récents : pip tente de compiler et échoue (`Compiler cl cannot compile programs`). L'environnement se crée toujours avec `py -3.11 -m venv .venv`.

Identité Git, PowerShell, comptes GitHub

Chaque membre, sur son poste :

```
git config --global user.name "Prénom Nom"
git config --global user.email "prenom.nom@ecole.ma"
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

- `git shortlog` compte ce nom : le TP1 exige deux auteurs distincts.
- `PS C:\...` est PowerShell ; `C:\...>` est cmd, où `Rename-Item` ou `Get-Content` n'existent pas.
- Compte de l'enseignant : `Larhlimiabd-ai`. Plusieurs comptes sur un poste : mettre le compte dans l'adresse (`https://<compte>@github.com/...`) ou `cmdkey /delete:git:https://github.com`.

Qui fait quoi dans le binôme

TP	Membre A	Membre B	À deux
TP1	Kit, environnement, dépôt GitHub privé, invite B et l'enseignant, premier push avec `requirements.lock`	Accepte l'invitation, clone, environnement, commit de sa ligne du `README.md`	Les trois fiches ; vérification finale ; clone frais
TP2	Installe le kit, génère, entraîne, évalue, pousse	Récupère, réentraîne, contrôle C5	Lecture des erreurs, `docs/semaine2.md`
TP3	Installe le kit, réentraîne, teste, pousse	Récupère, réentraîne, essais de l'API (T15 à T20)	`docs/semaine3.md`
TP4	Étapes B et C : contrat d'entrée, courbe, abstention	Étapes D et E : intégration, typage, ruff, pre-commit, CI	Jeu d'or (dix avis chacun), `docs/semaine4.md`
TP5	Installe le kit, lance l'API et la page, T29 à T31	Récupère, erreurs (T32), fichier d'avis (T33)	`docs/semaine5.md`, la CI
TP6	Kit, déplacement de `predict.py`, API branchée (T35 à T37)	Commande, roue, installation ailleurs (T38 à T40)	Journal des versions, étiquette, CI (T41)

`git pull` avant de commencer, un commit par tâche, `git push` à la fin. Chaque membre commite dans chaque TP, et réentraîne le modèle sur son poste après un `git pull`.

TP1 : cadrer l'application et préparer l'environnement

Étape 1 : récupérer le kit (membre A, T0)

Sur le dépôt du cours, « Code » puis « Download ZIP ». Pas de `git clone` : il emporterait l'historique de l'enseignant, que le test d'historique compterait.

```
New-Item -ItemType Directory -Force C:\IA-DEV
Expand-Archive "$HOME\Downloads\sentiment-app-main.zip" -DestinationPath C:\IA-DEV -Force
Rename-Item C:\IA-DEV\sentiment-app-main sentiment-app
cd C:\IA-DEV\sentiment-app
Get-ChildItem -Force
```

T0 : la structure est là

Les sept dossiers `app`, `data`, `docs`, `models`, `scripts`, `tests`, `ui`, plus `.github` et `.gitignore`.

Étape 2 : l'environnement (membre A)

```
py -3.11 -m venv .venv
.venv\Scripts\Activate.ps1
python --version
Select-String -Path requirements.txt -Pattern "^pyarrow"
```

Si une ligne pyarrow==21.0.* s'affiche, la retirer

Elle contredit `mlflow==2.17.*` : pip refuserait toute l'installation (`ResolutionImpossible`). Streamlit et MLflow déclarent déjà pyarrow.

```
(Get-Content requirements.txt) | Where-Object { $_ -notmatch '^pyarrow' } | Set-Content
    requirements.txt
```

Étape 2, suite : installer et mesurer le départ (T1)

```
python -m pip install --upgrade pip
pip install -r requirements.txt
python -m pip freeze | Out-File -Encoding utf8 requirements.lock
python scripts/check_setup.py
python -m pytest -q
```

T1 : le point de départ

`Python 3.11.x`. Installation de 5 à 10 minutes (tout le semestre, PyTorch compris). Le diagnostic signale encore une erreur et des avertissements. pytest : **3 failed, 31 passed, 4 skipped** ; les échecs réclament les fiches, les tests ignorés attendent le dépôt Git.

Équivalent en une commande : `powershell -ExecutionPolicy Bypass -File scripts\setup.ps1`.

Étape 3 : les fiches de cadrage (à deux, 50 min, T2)

Règle vérifiée, dans chacune des trois fiches	Test
Chaque rubrique, sauf « Approche retenue », contient au moins 20 caractères rédigés (hors consignes > et cases à cocher)	`test_sections_are_filled`
Une seule case cochée, écrite `- [x]` avec un x minuscule	`test_exactly_one_approach_checked`
La justification cite au moins deux critères : (a) données étiquetées, (b) vérifiabilité, (c) coût par requête, (d) conséquence d'une erreur	`test_justification_cites_two_criteria`

```
python -m pytest tests/test_cadrage.py -q
python -m pytest -q
```

T2 : les fiches sont complètes

12 passed, puis **34 passed, 4 skipped**. Modèle à ne pas recopier : `EXEMPLE-corrige.md`.

Étape 4 : le dépôt GitHub (membre A, T3)

Sur GitHub : dépôt **privé** `sentiment-app`, vide. Settings, Collaborators : inviter le membre B, puis l'enseignant (`Larhlimiabd-ai`).

```
git init
git add -A
git status
git commit -m "TP1 : environnement, structure et cadrage"
git branch -M main
git remote add origin https://github.com/<compte-A>/sentiment-app.git
git push -u origin main
python -m pytest -q
```

T3 : rien de lourd ni de secret dans Git

Ni `.venv/`, ni `.env`, ni `.joblib` dans `git status`. Première CI rouge (un seul auteur) : attendu. pytest : **37 passed, 1 failed**. Remplacer `<compte-A>` sans chevrons (sinon erreur 400).

Étape 5 : rejoindre le dépôt (membre B, T4)

```
cd C:\IA-DEV
git clone https://github.com/<compte-A>/sentiment-app.git sentiment-app
cd sentiment-app
py -3.11 -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python scripts/check_setup.py
notepad README.md
git add README.md
git commit -m "README : ajout de <nom> au tableau du binome"
git push
```

T4 : l'environnement du membre B

`0 erreur(s)` au diagnostic. Ne commitez que `README.md` : `requirements.lock` diffère d'un poste à l'autre, seul le membre A le commite.

Étape 6 : un commit par membre (membre A, T5)

```
git pull
notepad README.md
git add README.md
git commit -m "README : ajout de <nom> au tableau du binome"
git push
git shortlog -sn --all
python -m pytest tests/test_git_history.py -q
```

T5 : deux auteurs

Deux noms distincts dans `git shortlog`, puis **3 passed**.

Push refusé (non-fast-forward)

`git pull` ; en cas de conflit sur `README.md`, garder les deux lignes, retirer les marqueurs `<<<<<<<`, `=======`, `>>>>>>>`, puis `git add`, `git commit`, `git push`.

Étapes 7 et 8 : vérification finale (T6) et clone frais (T7)

```
git pull
python scripts/check_setup.py
python -m pytest -q
```

T6 : tout est vert

`0 erreur(s), 0 avertissement(s),` **38 passed**, onglet Actions vert.

```
cd $env:TEMP
git clone https://github.com/<compte-A>/
    sentiment-app.git verif
cd verif
py -3.11 -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python scripts/check_setup.py
python -m pytest -q
```

T7 : ailleurs que sur vos postes

`0 erreur(s)` et **38 passed**, sans intervention.

TP2 : entraîner, évaluer et sauvegarder le modèle

Installer le kit (C1)

```
cd C:\IA-DEV\sentiment-app
Expand-Archive "$HOME\Downloads\tp2-semaine2-kit.zip" -DestinationPath C:\IA-DEV -Force
python C:\IA-DEV\tp2-semaine2\appliquer_tp2.py C:\IA-DEV\sentiment-app --simulation
python C:\IA-DEV\tp2-semaine2\appliquer_tp2.py C:\IA-DEV\sentiment-app
python -m pytest -q
```

C1 : le point de départ du TP2

La simulation liste les scripts de `src/`, `tests/test_model.py`, le notebook, les documents du TP et l'ajout de `!models/metrics.json` au `.gitignore`. pytest : **2 failed, 38 passed, 2 skipped**. Les échecs réclament `metrics.json`, les tests ignorés attendent le modèle.

Générer, entraîner, évaluer (C2 à C4)

```
python src/make_demo.py
python src/train.py
git status --short
python src/evaluate.py
```

Attendu

- C2 : `290 avis écrits`, 145 positifs, 145 négatifs, dont 100 mitigés.
- C3 : `Vocabulaire : 316 termes`, `F1 macro ... : 0.879` ; `models/metrics.json` apparaît dans `git status`, ni le modèle ni `data/demo_reviews.csv`.
- C4 : matrice `27 2 / 5 24` : 7 erreurs sur 58, toutes sur des avis mitigés.

Un sac de mots compte les mots, il ne voit pas le retournement d'un avis mitigé : c'est l'objet de `docs/semaine2.md`, en trois phrases.

Le vrai jeu de données, les tests, le push (C5)

```
python src/train.py --allocine
python -m pytest -q
notepad docs\semaine2.md
git add -A
git commit -m "Semaine 2 : modele de sentiment, metriques et tests"
git push
```

C5 : tout est vert

`Source : allocine, 20000 avis, F1 macro ... : 0.920`, puis **42 passed** et la CI verte : elle n'a jamais le modèle, ses tests de métriques lisent `metrics.json`.

Membre B : `git pull`, `python src/make_demo.py`, `python src/train.py`, `python -m pytest -q` : 42 passed. Allociné injoignable : garder le jeu de démonstration et le signaler.

Étape F, complément : ce que regarde le modèle (C6)

```
python C:\IA-DEV\complements-lot1\appliquer_complements.py C:\IA-DEV\sentiment-app --etape tp2
python src/expliquer.py
python -m pytest tests/test_explicabilite.py -q
```

Les termes les plus influents

Vers positive : « je » +0,771, « que je », « en » +0,683. Vers negative : « cassé » −0,923, « au » −0,689. Des mots-outils : le modèle a appris des tournures du jeu de démonstration.

L'avis mitigé, expliqué par SHAP

`negative` à 0,5335. 20 termes vers negative (−1,006), 13 vers positive (+0,918), absents −0,035, base −0,011 : la somme redonne la sortie du modèle, −0,1343.

C6 : 4 passed

Additivité, calcul à la main (coefficient fois écart à la moyenne), ordre des termes, hésitation sous 0,60.

TP3 : exposer le modèle par une API

Installer le kit (T14)

```
cd C:\IA-DEV\sentiment-app
Expand-Archive "$HOME\Downloads\kit-tp3.zip" -DestinationPath C:\IA-DEV -Force
python C:\IA-DEV\kit-tp3\appliquer_tp3.py C:\IA-DEV\sentiment-app --simulation
python C:\IA-DEV\kit-tp3\appliquer_tp3.py C:\IA-DEV\sentiment-app
pip install -r requirements.txt
python src/train.py
python -m pytest -q
```

Le script copie `api/`, `tests/test_api.py`, `docs/semaine3.md` ; ajoute `pythonpath = .` à `pytest.ini` ; apprend à la CI à installer l'API et à entraîner le modèle. `train.py` remet le modèle de démonstration, celui de la CI.

T14 : 48 tests verts

48 passed : les 42 du TP2 et les six de l'API. `pip install` se lance dans le dépôt, jamais dans le kit.

Faire répondre l'API (T15 à T18)

```
uvicorn api.main:app --reload
```

Puis `http://localhost:8000/docs` : déplier la route, « Try it out », « Execute ».

Contrôle	Requête	Attendu
T15	`GET /health`	200, `"status": "OK"`, modèle `tfidf-logreg (TP2, ...)`
T16	`{"text": "Livraison rapide, produit conforme, très satisfait"}`	200, POSITIVE, 0,777
T17	`{"text": "Produit cassé et service injoignable, à fuir"}`	200, NEGATIVE, 0,7465
T18	`{"text": ""}`, puis `{}`, puis `{"text": 42}`	422 dans les trois cas

Pousser (T19), l'API sans modèle (T20)

```
git add -A
git commit -m "Semaine 3 : API FastAPI"
git push
```

T19 : la CI entraîne puis teste

L'étape « Modèle de démonstration » passe, puis « Tests » : 48 passed.

```
Rename-Item models/sentiment.joblib
    sentiment.bak
uvicorn api.main:app
Rename-Item models/sentiment.bak sentiment.
    joblib
```

T20 : refus de démarrer

`FileNotFoundError: models/sentiment.joblib absent`, puis `Application startup failed`.

`docs/semaine3.md` : pourquoi refuser de démarrer plutôt que répondre 503 ? Le double refus du texte vide (Pydantic, puis `predict()`) est-il redondant ? **Membre B** : `git pull`, réentraîner, 48 passed, puis les essais de l'API.

Les tests de l'API

Dans le navigateur : la page /docs

FastAPI génère cette page à partir du code. L'API doit tourner dans un autre terminal (`uvicorn api.main:app`).

Contrôle	Requête	Au TP3	Après le TP4
T15	`GET /health`	200, `"status": "OK"`	identique
T16	`{"text": "Livraison rapide, produit conforme, très satisfait"}`	200, `POSITIVE`, 0,777	identique
T17	`{"text": "Produit cassé et service injoignable, à fuir"}`	200, `NEGATIVE`, 0,7465	identique
T18	`{"text": ""}`, `{}`, `{"text": 42}`	422 trois fois	identique
T22	`{"text": "  "}` (trois espaces)	422, `detail` en chaîne	422, `detail` en liste
T24	`{"text": "xqzvb wlkptr grombu"}`	200, `NEGATIVE`, 0,5162	200, `INDETERMINE`, 0,5162

En ligne de commande, avec httpx

```
python -c "import httpx; print(httpx.get('http://127.0.0.1:8000/health').json())"
python -c "import httpx; r = httpx.post('http://127.0.0.1:8000/predict', json={'text': 'Livraison rapide, produit conforme,
       très satisfait'}); print(r.status_code, r.json())"
python -c "import httpx; print([httpx.post('http://127.0.0.1:8000/predict', json=c).status_code for c in ({'text': ''}, {},
       {'text': 42})])"
python -c "import httpx; r = httpx.post('http://127.0.0.1:8000/predict', json={'text': '   '}); print(r.status_code, r.json
      ())"
```

Sorties mesurées au TP3

```
{'status': 'OK', 'model': 'tfidf-logreg (TP2, models/sentiment.joblib)'}
200 {'label': 'POSITIVE', 'score': 0.777, 'text': 'Livraison rapide, produit conforme, très satisfait'}
[422, 422, 422]
422 {'detail': 'texte vide'}
```

Après le TP4, la dernière ligne donne un `detail` en liste : `[{'type': 'value_error', 'loc': ['body', 'text'], ...}]`.

Les tests automatiques du TP3

`TestClient` appelle l'API en mémoire, sans serveur. Les tests sont ignorés sans modèle ; la CI l'entraîne avant de les lancer.

Test de `tests/test_api.py`	**Ce qu'il vérifie**
`test_root`	`GET /` répond 200 et indique `/docs`
`test_health`	`status` vaut `OK`, le modèle est celui du TP2
`test_predict_positif`	Un avis évident reçoit `POSITIVE`, score d'au moins 0,5
`test_predict_negatif`	Un avis évident reçoit `NEGATIVE`
`test_predict_texte_vide_422`	`""` et `"  "` reçoivent 422
`test_predict_champ_manquant_422`	`{}` et `{"text": 42}` reçoivent 422

```
python -m pytest -q tests/test_api.py
python -m pytest -v tests/test_api.py
python -m pytest -q --cov=api --cov-report=term-missing
```

Les tests du TP4, et la lecture d'un 422

Fichier	Ce qu'il vérifie
`tests/test_limites.py`	Huit cas aux frontières du contrat, avec la forme de chaque 422 ; deux propriétés hypothesis (texte non blanc : 200 et étiquette du contrat ; texte de blancs : 422)
`tests/test_integration.py`	Démarrage réel ; refus de démarrer sans modèle ; 503 modèle retiré ; 500 sans message interne quand le modèle plante ; refus d'étiquettes inconnues
`tests/test_jeu_or.py`	Les 20 avis : aucune erreur franche ; abstentions comptées ; erreurs connues en `xfail(strict=True)`

Lire un refus 422

`detail` est normalement une liste d'objets : `loc` désigne le champ (`["body", "text"]`), `msg` explique, `type` classe. Au TP3, un texte d'espaces passe le schéma puis `predict()` le refuse : le 422 porte une simple chaîne. Deux formes pour un même refus : premier défaut que le TP4 fait trouver.

TP4 : tests approfondis et qualité

Avant la séance : le jeu d'or

Vingt avis rédigés par le binôme (dix chacun), absents des données d'entraînement, avec l'étiquette attendue. Au moins cinq difficiles : négation, ironie, avis mitigé, autre domaine. Une erreur que le modèle ne sait pas éviter reste dans le jeu, avec sa raison dans `erreur_connue`.

```
text,attendu,difficulte,erreur_connue
"Commande reçue en deux jours, emballage soigné",POSITIVE,,
"Pas déçu du tout, je recommande",POSITIVE,négation,
"Arrivé cassé, bravo le transporteur",NEGATIVE,ironie,
```

Le TP4 en une phrase

Il n'ajoute presque rien à l'API : il cherche ses défauts, les corrige et installe les garde-fous. Seul changement visible, un troisième verdict : `INDETERMINE`.

Étape A : installer le kit (T21)

```
cd C:\IA-DEV\sentiment-app
Expand-Archive "$HOME\Downloads\tp4-tests-qualite-kit.zip" -DestinationPath C:\IA-DEV -Force
python C:\IA-DEV\tp4-tests-qualite\appliquer_tp4.py C:\IA-DEV\sentiment-app --simulation
python C:\IA-DEV\tp4-tests-qualite\appliquer_tp4.py C:\IA-DEV\sentiment-app
pip install -r requirements-dev.txt
python src/train.py
python -m pytest -q
```

T21 : le point de départ

1 failed, 51 passed, 1 skipped. L'échec, `test_jeu_or_complet`, réclame le jeu d'or (« 0 avis sur 20 ») ; le test sauté attend ces mêmes avis.

Étape A : mesurer le départ, sans rien corriger

```
python -m pytest -q --cov=api --cov-report=term-missing
ruff check .
mypy api
```

Les trois mesures

- Couverture : 91 %, six lignes de `api/main.py` : 49 (le 503), 77 à 79 (le 500), 85 à 87 (le lancement direct).
- ruff : 11 remarques. B904 (2) dans `main.py` ; I001 et F401 dans `model.py` ; F401 et B905 (2) dans `evaluate.py` ; I001 dans `test_api.py` ; E741 (3) dans deux tests du TP1.
- mypy : `Success: no issues found in 4 source files`.

Ces mesures vont dans `docs/semaine4.md`, parties 1 et 5.

Étape B : le contrat d'entrée (T22, T23)

Compléter `tests/test_limites.py` : huit cas au moins (vide, espaces, un caractère, 2000 et 2001 caractères, champ absent, 42, corps non JSON) avec la forme de chaque 422, puis deux propriétés hypothesis.

```
python -m pytest -q tests/test_limites.py
```

Avant la correction : deux échecs voulus

2 failed, 8 passed. Le cas des espaces reçoit 422 sous une autre forme ; hypothesis donne son plus petit contre-exemple : `texte='\x85'`, que `str.strip()` retire.

Corriger le *schéma* (`field_validator`, même règle que `predict()`), puis relancer trois fois.

T22 et T23

10 passed trois fois ; dans `/docs`, les espaces reçoivent un 422 dont `detail` est une liste.

Étape C : l'abstention (T24)

Trois textes de mots inventés reçoivent tous `NEGATIVE` à 0,5162 : sans mot connu, le vecteur TF-IDF est nul, il ne reste que l'ordonnée à l'origine.

```
python src/courbe_seuil.py
```

La courbe du seuil

```
 seuil  répond à   part  exactitude  erreurs  vers un humain
  0.50        58   100%       0.879        7               0
```

À 0,50, on retrouve le TP2 ; sinon, relancer `python src/train.py`.

Dans `api/model.py` : sous `SEUIL_ABSTENTION` (0,60 par défaut), l'étiquette devient `INDETERMINE`, le score reste renvoyé. **T24** : un texte inventé reçoit `INDETERMINE`.

Étape C : le jeu d'or (T25)

Copier les vingt avis dans `data/jeu_or.csv`, puis :

```
python src/bilan_jeu_or.py
python -m pytest -q tests/test_jeu_or.py
```

T25 : le jeu d'or est vert

Le bilan compte les réponses justes, les abstentions et les erreurs déclarées : il va dans `docs/semaine4.md`. Aucun `failed` ; les erreurs déclarées apparaissent en `xfailed`.

Ne pas tricher avec le jeu d'or

Retirer un avis parce que le modèle s'y trompe vide le jeu de son sens. On le garde et on déclare l'erreur ; `strict=True` signalera le jour où elle sera corrigée.

Étape D : l'API telle qu'elle tourne (T26)

Écrire `tests/test_integration.py` : démarrage réel (`with TestClient(app)`), modèle absent (`monkeypatch`, puis `charger_modele.cache_clear()`), modèle retiré de `app.state`, modèle en panne (`RuntimeError`, puis `ValueError("bug interne")`).

```
python -m pytest -q tests/test_integration.py
python -m pytest -q --cov=api --cov-report=term-missing --cov-fail-under=95
```

T26

Avant correction, la panne `ValueError` reçoit 422 avec le message interne : c'est le défaut. Une fois la conversion en 422 retirée de `main.py`, toute panne répond 500, et la sortie finit par `Required test coverage of 95% reached` (97 % sans exclusion, 100 % avec `__main__` exclu et justifié).

Étape E : le contrat typé et ruff

Le contrat typé. mypy ne dit rien : `predict()` renvoie un `dict`, d'où ne sortent que des `Any`. Avec un `TypedDict` et `Literal["POSITIVE", "NEGATIVE", "INDETERMINE"]`, mypy signale `.upper()` : le remplacer par une correspondance explicite, vérifiée au démarrage.

ruff. `ruff check . --fix`, puis à la main : `strict=True` dans les `zip()` de `evaluate.py`, `from None` sur le `raise` du 500, et `l` renommé `ligne` dans deux tests du TP1 :

Fichier, ligne	Correction
`test_cadrage.py`, 51	`l for l in body.splitlines() if l.strip() ...` → `ligne for ligne in body.splitlines() if ligne.strip() ...`
`test_environment.py`, 38	`[l.strip() for l in` → `[ligne.strip() for ligne in`
`test_environment.py`, 39	`[l for l in lines if l and not l.startswith("#")]` → `[ligne for ligne in lines if ligne and not ligne.startswith("#")]`

Étape E : le crochet pre-commit

```
pre-commit install
Add-Content src\courbe_seuil.py "import os"
git add src\courbe_seuil.py
git commit -m "essai"
git restore --staged src\courbe_seuil.py
git restore src\courbe_seuil.py
```

Attendu

La ligne `ruff` passe en `Failed` et le commit n'est pas créé. Les deux dernières commandes annulent l'essai.

Le crochet s'installe dans chaque clone : le membre B lance aussi `pre-commit install` sur son poste.

Étape E : la CI, et le contrôle final (T27)

Dans `.github\workflows\ci.yml`, remplacer l'étape « Tests » par les quatre étapes de `ci-tp4.yml`, retrait de six espaces ; chacune active l'environnement, comme au TP1 :

```
      - name: Outils de qualité (TP4)
        run: . .venv/bin/activate && pip install -r requirements-dev.txt
      - name: ruff
        run: . .venv/bin/activate && ruff check .
      - name: mypy
        run: . .venv/bin/activate && mypy api
      - name: Tests et couverture
        run: . .venv/bin/activate && pytest -q --cov=api --cov-report=term-missing --cov-fail-under=95
```

T27 : tout est vert, en local et en CI

`ruff check .` : `All checks passed!` ; `mypy api` : `Success` ; `python -m pytest -q` : aucun `failed` (83 passed, 2 xfailed avec le corrigé). Push : les étapes ruff, mypy, couverture passent.

Étape F, complément : le modèle sert-il tous les avis ? (T27a)

```
python C:\IA-DEV\complements-lot1\appliquer_complements.py C:\IA-DEV\sentiment-app --etape tp4
python src/biais.py
```

Groupe	Avis	Exactitude	Erreurs	Vers humain	Restantes
Avec retournement	17	0,588	7	15	2
Direct	41	1,000	0	1	0
Long (plus de 9 mots)	27	0,741	7	16	2
Court (9 mots ou moins)	31	1,000	0	0	0
Long et direct	10	1,000	0	1	0

La longueur est une variable confondante

Les avis longs semblent moins bien servis, mais les 10 avis longs sans retournement sont tous justes : c'est le retournement qui compte. Puis `8 passed`.

TP5 : l'interface Streamlit

Le rôle de l'interface, et comment fonctionne Streamlit

Un client de l'API, comme un autre

La page du service client : on y tape un avis, ou on charge un fichier d'avis, et on lit le verdict. Elle envoie `POST /predict` et traduit chaque réponse en message clair, abstention et erreurs comprises. Elle n'importe jamais `predict()` ni le `.joblib` : un modèle réentraîné ne change rien à l'interface.

Un script réexécuté à chaque action

Une application Streamlit est un script Python ordinaire, relancé de haut en bas à chaque clic. Les composants (`st.text_area`, `st.button`, `st.file_uploader`) renvoient leur valeur ; les affichages (`st.metric`, `st.warning`, `st.error`) écrivent dans la page.

Deux programmes tournent : l'API sur le port 8000, la page sur le port 8501.

Installer le kit (T28)

```
cd C:\IA-DEV\sentiment-app
Expand-Archive "$HOME\Downloads\tp5-interface-kit.zip" -DestinationPath C:\IA-DEV -Force
python C:\IA-DEV\tp5-interface\appliquer_tp5.py C:\IA-DEV\sentiment-app --simulation
python C:\IA-DEV\tp5-interface\appliquer_tp5.py C:\IA-DEV\sentiment-app
python src/train.py
python -m pytest -q
```

T28 : douze tests de plus, aucun échec

La simulation liste `ui/`, `tests/test_client.py`, `tests/test_interface.py`, `data/avis_exemple.csv`, `docs/semaine5.md`, puis l'exception `!data/avis_exemple.csv` et Streamlit dans la CI. pytest : aucun `failed`, douze tests de plus (95 passed et 2 xfailed avec le corrigé du TP4).

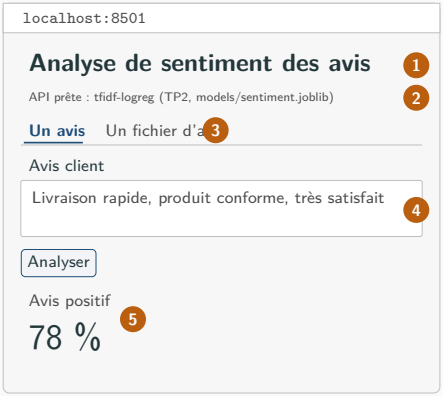
Trois fichiers, trois rôles

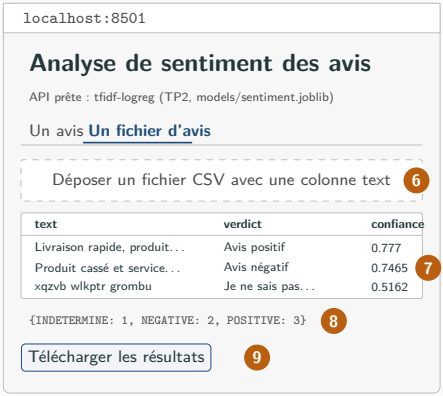
Fichier	Rôle
`ui/client.py`	Tout ce que la page sait de l'API : santé, prédiction, lot, résumé
`ui/app.py`	La page : un onglet pour un avis, un pour un fichier
`test_client.py`	Sept tests du client
`test_interface.py`	Cinq tests de la page (`AppTest`)

Les tests remplacent la connexion réelle par un `TestClient` : l'API tourne en mémoire, sans serveur.

```
def predire(session: httpx.Client, texte: str) -> Reponse:
    try:
        r = session.post("/predict", json={"text": texte})
    except httpx.TransportError:
        return Reponse(False, message=INJOIGNABLE)
    if r.status_code == 200:
        corps = r.json()
        return Reponse(True, label=corps["label"],
                       score=corps["score"])
    if r.status_code == 422:
        return Reponse(False, message=REFUS)
    if r.status_code == 503:
        return Reponse(False, message=SANS_MODELE)
    return Reponse(False, message=f"Erreur interne de l'API ({r.
      status_code}).")
```

La page, zone par zone



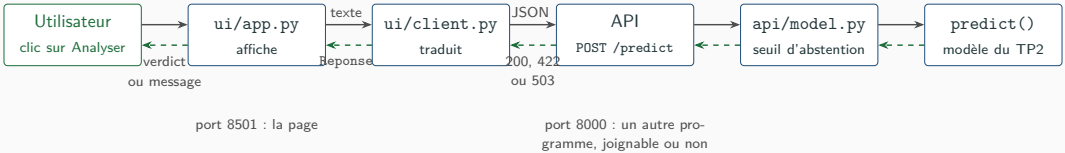


À gauche, l'onglet « Un avis » après une analyse ; à droite, l'onglet « Un fichier d'avis » après le chargement de `data/avis_exemple.csv`.

Chaque zone et son code

Zone	Ce que voit l'utilisateur	Appel Streamlit	Fonction du client
1	Le titre	`st.title`	
2	L'état de l'API, vérifié à chaque réexécution	`st.caption`, ou `st.error`	`sante()`
3	Les deux onglets	`st.tabs`	
4	Le champ de l'avis et le bouton	`st.text_area`, `st.button`	
5	Verdict et confiance ; abstention, refus ou erreur	`st.metric`, `st.info`, `st.warning`, `st.error`	`predire()`
6	Le dépôt du fichier CSV	`st.file_uploader`	
7	Le tableau des verdicts	`st.dataframe`	`predire_lot()`
8	Le résumé par verdict	`st.write`	`resumer()`
9	Le téléchargement des résultats	`st.download_button`	

Le chemin d'une requête



Un clic traverse l'application, puis la réponse revient traduite à chaque étage. La page et l'API sont deux programmes : le client transforme chaque cas (API arrêtée, modèle absent, texte refusé) en message, pour que la page n'interprète jamais un code HTTP.

Le code de la page (1/3) : imports, page, état de l'API

Bloc 1 : les imports

```
import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1])
      )  # pour importer ui.client

import pandas as pd
import streamlit as st

from ui import client
```

Bloc 2 : la page et l'état de l'API

```
st.set_page_config(page_title="Analyse de sentiment")
st.title("Analyse de sentiment des avis")

session = client.creer_session()
etat = client.sante(session)
if etat.ok:
    st.caption(f"API prête : {etat.message}")
else:
    st.error(etat.message)
```

Streamlit ajoute au chemin le dossier `ui/`, pas la racine du dépôt : d'où `sys.path.insert`. La connexion passe par `creer_session()`, le point que les tests remplacent. `sante()` interroge `/health` à chaque réexécution.

Le code de la page (2/3) : l'onglet « Un avis »

```
onglet_avis, onglet_lot = st.tabs(["Un avis", "Un fichier d'avis"])

with onglet_avis:
    texte = st.text_area("Avis client")
    if st.button("Analyser"):
        reponse = client.predire(session, texte)
        if reponse.ok and reponse.label is not None and reponse.score is not None:
            st.metric(client.VERDICTS[reponse.label], f"{reponse.score:.0%}")
            if reponse.label == "INDETERMINE":
                st.info("La confiance est sous le seuil d'abstention : un humain doit trancher.")
        elif reponse.message == client.REFUS:
            st.warning(reponse.message)
        else:
            st.error(reponse.message)
```

Le bouton déclenche `predire()`, qui renvoie une `Reponse` : le verdict et son pourcentage, un encadré de plus pour l'abstention, un avertissement pour un texte refusé, une erreur pour le reste.

Le code de la page (3/3) : l'onglet « Un fichier d'avis »

```
with onglet_lot:
    fichier = st.file_uploader("Fichier CSV avec une colonne text", type="csv")
    if fichier is not None:
        avis = pd.read_csv(fichier)
        if "text" not in avis.columns:
            st.error("Colonne text absente du fichier.")
        else:
            reponses = client.predire_lot(session, avis["text"].astype(str).tolist())
            avis["verdict"] = [client.VERDICTS[r.label] if r.ok and r.label else r.message for r in reponses]
            avis["confiance"] = [r.score for r in reponses]
            st.dataframe(avis, use_container_width=True)
            st.write(client.resumer(reponses))
            st.download_button("Télécharger les résultats", avis.to_csv(index=False),
                               file_name="resultats.csv", mime="text/csv")
```

Sans colonne `text`, la page le dit et s'arrête. Sinon, `predire_lot()` envoie chaque avis à l'API, deux colonnes s'ajoutent, `resumer()` compte les verdicts, et le tableau complété se télécharge.

Le cycle de Streamlit, et les tests

À chaque action, tout le script repart du haut

- La santé de l'API est revérifiée à chaque fois : son arrêt se voit dès le rechargement.
- Le texte saisi et le fichier déposé gardent leur valeur d'une exécution à l'autre.
- Le lot est recalculé à chaque réexécution, même au clic sur « Télécharger » : coûteux pour dix mille avis ; `@st.cache_data` est la piste d'amélioration.

Comment les tests se branchent

`test_client.py` passe au client un `TestClient` (l'API en mémoire). `test_interface.py` remplace `client.creer_session` par ce `TestClient`, exécute la page avec `AppTest`, saisit, clique, puis lit `page.metric`, `page.warning`, `page.error`. Aucun serveur à lancer.

Lancer et analyser (T29 à T31)

```
uvicorn api.main:app --reload
```

```
streamlit run ui/app.py
```

Deux terminaux ; au premier lancement, Entrée pour passer la demande d'adresse ; page sur `http://localhost:8501`.

Contrôle	Ce qu'on fait	Ce que la page affiche
T29	On ouvre la page	`API prête : tfidf-logreg (TP2, models/-sentiment.joblib)`
T30	« Livraison rapide, produit conforme, très satisfait »	Avis positif, 78 %
T30	« Produit cassé et service injoignable, à fuir »	Avis négatif, 75 %
T31	« xqzvb wlkptr grombu »	Je ne sais pas : à faire vérifier par un humain, 52 % ; message sur le seuil d'abstention

Les erreurs (T32) et un fichier d'avis (T33)

T32 : chaque erreur a son message

Trois espaces : « Texte refusé : vide, fait d'espaces, ou de plus de 2000 caractères » (réponse 422). L'API arrêtée (`Ctrl+C` dans son terminal), puis la page rechargée : « API injoignable : lancez d'abord uvicorn api.main :app ». Relancer l'API ensuite.

T33 : le lot

Onglet « Un fichier d'avis », fichier `data/avis_exemple.csv` : un tableau de six avis avec verdict et confiance, le résumé `{'INDETERMINE': 1, 'NEGATIVE': 2, 'POSITIVE': 3}`, et « Télécharger les résultats », qui produit `resultats.csv`. Un avis sur six part vers un humain.

Compte rendu, push, CI (T34)

Les trois questions de `docs/semaine5.md` : pourquoi l'interface n'importe jamais le modèle ; ce qu'elle montre quand il hésite, et pourquoi c'est mieux qu'un verdict forcé ; ce que voit l'utilisateur quand l'API tombe.

```
git add -A
git commit -m "Semaine 5 : interface Streamlit"
git push
```

T34 : la CI est verte

La CI installe Streamlit et rejoue les douze tests du TP5. Le message pip `streamlit 1.39.1 requires packaging<25` est sans conséquence : les tests de l'interface passent.

Membre B : `git pull`, `python src/train.py`, `python -m pytest -q`, puis T29 à T33.

Étape G, complément : deux graphiques (T34a)

```
python C:\IA-DEV\complements-lot1\appliquer_complements.py C:\IA-DEV\sentiment-app --etape tp5
python -m pytest tests/test_graphiques.py tests/test_interface.py -q
```

Le script ajoute cinq lignes sous le résumé, dans `ui/app.py` :

```
colonne_verdicts, colonne_confiances = st.columns(2)
colonne_verdicts.caption("Répartition des verdicts")
colonne_verdicts.bar_chart(graphiques.repartition(reponses))
colonne_confiances.caption("Confiance du modèle, par tranche")
colonne_confiances.bar_chart(graphiques.histogramme_confiances(reponses))
```

T34a : 9 passed

Avec `avis_exemple.csv` : positif 3, négatif 2, à vérifier 1, refus 0 ; confiances 1, 1, 3, 1 et 0 avis de 50 à 100 %. Les calculs restent dans `ui/graphiques.py` : deux fonctions pures, testées sans Streamlit.

TP6 : packager l'application

Le TP6 en bref : un paquet installable

L'objectif

Faire du dépôt un paquet Python : une roue (`.whl`) qu'on installe avec `pip`, qui fournit la commande `sentiment`, et qui fonctionne hors du dépôt.

1. **Ce qui entre dans un paquet** : le code, mais ni les tests, ni les données, ni le modèle. Le paquet trouve le modèle grâce à la variable `SENTIMENT_MODELE`.
2. **Des imports propres** : l'API importe `sentiment_app.predict`, sans modifier `sys.path`.
3. **Une seule source pour la version** : `0.6.0`, lue par `pyproject.toml`, affichée par `sentiment --version`, gravée dans l'étiquette Git `v0.6.0`.
4. **Des dépendances déclarées**, scikit-learn épinglé : un modèle ne se relit qu'avec la version qui l'a produit.

Étape A : installer le kit (T35)

```
Expand-Archive "$HOME\Downloads\tp6-packaging-kit.zip" -DestinationPath C:\IA-DEV -Force
python C:\IA-DEV\tp6-packaging\appliquer_tp6.py C:\IA-DEV\sentiment-app --simulation
python C:\IA-DEV\tp6-packaging\appliquer_tp6.py C:\IA-DEV\sentiment-app
pip install -r requirements-dev.txt
python -m pytest -q
```

Le kit ajoute `sentiment_app/` (version et commande), six tests, `CHANGELOG.md` et `docs/semaine6.md`. Il complète `pyproject.toml`, `requirements-dev.txt` (`build`), `pytest.ini` (`pythonpath = . src`) et la CI.

T35 : deux échecs voulus, tous les autres tests passent

`test_le_modele_se_place_ou_l_on_veut` : le paquet n'a pas encore `predict`.

`test_l_api_ne_bricole_plus_le_chemin_de_python` : l'API modifie encore `sys.path`.

Étape B : predict.py entre dans le paquet (T36)

```
git mv src/predict.py sentiment_app/predict.py
Copy-Item C:\IA-DEV\tp6-packaging\extraits\predict_compatibilite.py src\predict.py
notepad sentiment_app\predict.py
```

Dans `sentiment_app/predict.py`, le modèle devient configurable :

```
RACINE = Path(__file__).resolve().parents[1]

def chemin_modele() -> Path:
    """SENTIMENT_MODELE si elle est définie, sinon le modèle entraîné dans le dépôt."""
    return Path(os.environ.get("SENTIMENT_MODELE") or RACINE / "models" / "sentiment.joblib")

MODELE = chemin_modele()
```

T36

`python -m pytest -q` : 1 failed, celui de l'API. La commande du TP2 marche toujours : `python src/predict.py "..."` affiche `positive (confiance 78%)`.

Le fichier de compatibilité, src/predict.py

```
import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))  # la racine
from sentiment_app import predict as _module  # noqa: E402

if __name__ == "__main__":
    raise SystemExit(_module.main())
sys.modules[__name__] = _module
```

Pourquoi il est indispensable

Les tests du TP2 et du TP4 écrivent encore `import predict`. À l'import, ce fichier se remplace par le vrai module (`sys.modules`) : les deux noms désignent le même objet. Le test « modèle absent » du TP4, qui modifie `predict.MODELE`, modifie donc bien le modèle que l'API utilise. `python src/predict.py "..."` reste valable.

Étape C : l'API importe le paquet (T37)

Avant, depuis le TP3

```
import sys
from pathlib import Path

RACINE = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(RACINE / "src"))
from predict import charger_modele  # noqa: E402
from predict import predict as _predict  # noqa: E402
```

Après

```
from sentiment_app.predict import charger_modele
from sentiment_app.predict import predict as _predict
```

```
python -m pytest -q
Select-String -Path api\*.py -Pattern "sys.path"
ruff check .
mypy api
```

T37 : tout vert

Aucun `failed`, six tests de plus qu'au TP5 ; `Select-String` ne trouve rien ; ruff et mypy ne signalent rien.

Étape D : la commande sentiment (T38)

```
pip install -e .
sentiment --version
sentiment predict "Livraison rapide, produit conforme, très satisfait"
sentiment predict "xqzvb wlkptr grombu"
sentiment serve
```

T38 : sorties mesurées

```
sentiment-app 0.6.0
{"label": "POSITIVE", "score": 0.777, "text": "Livraison rapide, produit conforme, très satisfait"}
{"label": "INDETERMINE", "score": 0.5162, "text": "xqzvb wlkptr grombu"}
```

`sentiment serve` lance l'API : `/health` répond `OK`. Dans un autre terminal, `sentiment ui` lance la page.

`-e` installe en mode développement : le paquet pointe vers vos fichiers. `predict` passe par `api/model.py`, comme l'API : le seuil d'abstention s'applique aussi en ligne de commande.

Étape E : la roue (T39)

```
Remove-Item -Recurse -Force dist -ErrorAction SilentlyContinue
python -m build --wheel
```

T39 : la roue et ce qu'elle déclare

`Successfully built sentiment_app-0.6.0-py3-none-any.whl`. Elle contient `api`, `sentiment_app`, `ui` et ses métadonnées : ni `tests`, ni `data`, ni `models`, ni `src`.

```
Requires-Dist: fastapi<0.116,>=0.115
Requires-Dist: uvicorn<0.31,>=0.30
Requires-Dist: pydantic<3,>=2.9
Requires-Dist: scikit-learn<1.6,>=1.5
Requires-Dist: joblib<2,>=1.4
Requires-Dist: streamlit<1.40,>=1.39; extra == "ui"
```

La roue vient de `pyproject.toml` : `packages = ["sentiment_app", "api", "ui"]`, et la version lue dans `sentiment_app.__version__`.

Étape E : la roue, hors du dépôt (T40)

```
cd $env:TEMP
py -3.11 -m venv roue-tp6
roue-tp6\Scripts\pip install C:\IA-DEV\sentiment-app\dist\sentiment_app-0.6.0-py3-none-any.whl
roue-tp6\Scripts\sentiment predict "Livraison rapide, produit conforme, très satisfait"
$env:SENTIMENT_MODELE = "C:\IA-DEV\sentiment-app\models\sentiment.joblib"
roue-tp6\Scripts\sentiment predict "Livraison rapide, produit conforme, très satisfait"
Remove-Item Env:SENTIMENT_MODELE
```

T40 : ailleurs que dans le dépôt

- `pip` installe la roue et ses dépendances, lues dans ses métadonnées.
- Sans la variable : `modèle introuvable : ...\site-packages\models\sentiment.joblib. Lancez python src/train.py, ou indiquez son chemin dans la variable SENTIMENT_MODELE`, code de sortie 1.
- Avec : `{"label": "POSITIVE", "score": 0.777, ...}`.

Pourquoi scikit-learn est épinglé

Mesuré pendant la vérification du kit

Avant le TP6, la CI installait scikit-learn sans version : elle entraînait avec la 1.9.1. La roue, elle, installe la 1.5.2. Relu par la roue, ce modèle provoque :

```
AttributeError: 'LogisticRegression' object has no attribute 'multi_class'
```

La correction, faite par appliquer_tp6.py

`pyproject.toml` déclare `scikit-learn>=1.5,<1.6`, et la CI installe `scikit-learn==1.5.*` : le modèle est entraîné et relu avec la même version.

Un modèle sauvegardé est lié à la bibliothèque qui l'a produit : c'est une dépendance entre un code et un artefact, que le TP7 (MLflow) apprendra à suivre.

Étape F : version, étiquette, CI (T41)

```
git add -A
git commit -m "Semaine 6 : paquet installable, version 0.6.0"
git tag -a v0.6.0 -m "TP6 : paquet installable"
git push
git push origin v0.6.0
```

La CI gagne une étape, ajoutée par le kit :

```
      - name: Roue installée hors du dépôt (TP6)
        run: |
          . .venv/bin/activate && python -m build --wheel
          python -m venv /tmp/roue && /tmp/roue/bin/pip install dist/*.whl
          cd /tmp && SENTIMENT_MODELE="$GITHUB_WORKSPACE/models/sentiment.joblib" /tmp/roue/bin/sentiment predict "
      Livraison rapide, produit conforme, très satisfait"
```

T41

Actions vert, la dernière étape se terminant par la prédiction `POSITIVE` ; l'étiquette `v0.6.0` visible sur GitHub.

Ce que le TP6 change dans le dépôt

Fichier	Changement
`sentiment_app/__init__.py`, `__main__.py`, `cli.py`	Nouveaux : la version `0.6.0`, la commande `sentiment`
`sentiment_app/predict.py`	Déplacé depuis `src/` par `git mv`, son historique compris ; modèle configurable
`src/predict.py`	Le fichier de compatibilité
`api/model.py`	Plus de `sys.path` : deux imports depuis `sentiment_app.predict`
`pyproject.toml`	`[build-system]`, `[project]`, les dépendances, la commande, la version lue dans le paquet
`pytest.ini`, `requirements-dev.txt`	`pythonpath = . src` ; l'outil `build`
`.github/workflows/ci.yml`	scikit-learn épinglé ; l'étape de la roue
`CHANGELOG.md`, `docs/semaine6.md`	Le journal des versions, de 0.1.0 à 0.6.0 ; le compte rendu
`tests/test_packaging.py`	Six tests : 103 au total
`dist/`, `*.egg-info/`	La roue et les fichiers de construction, hors de Git

Les pièges du TP6

Piège	Conséquence	Correction
Oublier le fichier de compatibilité	`No module named 'predict'` dans les tests du TP2 et du TP4	Le recréer depuis `extraits/`
`pip install .` sans `-e` pendant le développement	Le paquet installé est une copie figée	`pip install -e .`
Une ancienne roue dans `dist/`	On installe autre chose que ce qu'on croit	Vider `dist/` avant de construire
`SENTIMENT_MODELE` absente hors du dépôt	« modèle introuvable », code 1	Indiquer le chemin du modèle
Deux versions de scikit-learn	`AttributeError` au chargement	Entraîner et relire en 1.5
`git push` seul	L'étiquette reste sur le poste	`git push origin v0.6.0`

Étape G, complément : la fiche modèle (T41a)

```
python C:\IA-DEV\complements-lot1\appliquer_complements.py C:\IA-DEV\sentiment-app --etape tp6
python -m pytest tests/test_fiche_modele.py -q
notepad docs\fiche-modele.md
```

Dix rubriques

Identité, usage prévu, usages exclus, données d'entraînement, performances mesurées, limites connues, biais mesurés, explicabilité, conditions d'usage, maintenance.

T41a

`13 failed, 1 passed` au départ, voulus ; `14 passed` une fois la fiche remplie : version 0.6.0, F1 0,879 et rappel des négatifs 0,931 lus dans `metrics.json`, un humain prévu dans les conditions d'usage.

La fiche suit le modèle

Réentraîner avec `--allocine` sans mettre la fiche à jour fait échouer le test des performances.

Vérification et notation par l'enseignant

L'accès aux dépôts, et la CI en continu

L'accès

Chaque dépôt est privé. Le membre A invite `Larhlimiabd-ai` (Settings, Collaborators). L'enseignant accepte depuis le courriel ou `https://github.com/<compte-A>/<depot>/invitations` (sept jours). Sur son poste, l'adresse porte son compte : `https://Larhlimiabd-ai@github.com/<compte-A>/<depot>.git`.

La CI, en continu

Chaque push est testé sur une machine neuve. L'onglet Actions montre où en est chaque binôme sans rien installer ; Insights, puis Contributors, montre les deux membres.

La relève automatique, à la date de remise

```
python releve_binomes.py binomes.csv --tp 3 --tests
python releve_binomes.py binomes.csv --tp 4 --tests --date-limite "2026-10-25 23:59"
python releve_binomes.py binomes.csv --tp 5 --tests
python releve_binomes.py binomes.csv --tp 6 --tests
```

`binomes.csv` : colonnes `binome` et `depot`. Le script reclone chaque dépôt, se place sur le dernier commit avant la limite, compte les commits tardifs, et écrit `releve/releve-tpN.md` et `.csv`.

Colonne	Ce qu'elle dit
`auteurs`	Nombre et noms des auteurs
`fichiers interdits`	`.venv`, modèle, `data/demo_reviews.csv` ou `.env` versionnés
`livrables`	Fiches, `metrics.json`, comptes rendus encore en canevas, jeu d'or incomplet, pre-commit
`tests du kit retouchés`	Tests fournis modifiés depuis leur ajout
`F1, pytest, couverture, ruff, mypy`	Avec `--tests` : entraînement et tests rejoués

Lire la colonne « tests du kit retouchés »

Un test vert ne prouve rien si le test a été modifié

L'essai du script sur un dépôt fautif l'a montré : un seuil de F1 abaissé dans `test_model.py` faisait passer pytest. Chaque fichier signalé se lit avec `git log -p -- tests/<fichier>`, dans le clone de `releve\`.

Retouches légitimes au TP4

L'ordre des imports de `test_api.py` remis par ruff ; ses étiquettes mises à jour après l'abstention ; le renommage de `l` en `ligne` dans `test_cadrage.py` et `test_environment.py`.

Avec `--tests`, le script exécute le code des étudiants : le lancer dans l'environnement du cours.

Le clone frais, à la main

```
cd $env:TEMP
git clone https://Larhlimiabd-ai@github.com/<
     compte-A>/<depot>.git verif
cd verif
py -3.11 -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python scripts/check_setup.py
python src/make_demo.py
python src/train.py
python -m pytest -q
git shortlog -sn --all
git log --format="%h %ad %an : %s" --date=short
git log --all --oneline -- models/sentiment.joblib
      .venv
```

Ce que l'enseignant doit voir

`0 erreur(s)` ; F1 0,879 ; le compte de tests du TP (48 au TP3) ; deux noms ; chaque membre présent dans chaque TP ; rien pour la dernière commande. Puis `uvicorn api.main:app` et T15 à T20. Au TP4 : `requirements-dev.txt`, `ruff`, `mypy`, la couverture.

Gardez cet environnement : il contient tout le semestre et sert à la relève des autres binômes.

Les grilles : TP1 et TP2 (sur 10)

TP	Critère	Pts	Vérification
TP1	Fiches justifiées par deux critères, rubriques remplies	4	Lecture, `test_cadrage.py`
	Dépôt clonable, installation sans erreur	2	Clone frais
	Structure et `.gitignore` conformes, `.venv/` absent de l'historique	2	`test_environment.py`, `git log`
	Un commit par membre, messages explicites	2	`git shortlog`, `git log`
TP2	Modèle entraîné, sauvegardé, rechargeable par `predict()`	3	Clone frais, `src/predict.py`
	Évaluation correcte : découpage avant vectorisation, mesures sur le jeu de test	3	Code, `metrics.json`
	Les 4 tests du modèle, CI verte	2	pytest, Actions
	Analyse des erreurs	2	`docs/semaine2.md`

Les grilles : TP3 et TP4 (sur 10)

TP	Critère	Pts	Vérification
TP3	L'API répond : `/health`, avis positif, avis négatif	3	T15 à T17
	Codes d'erreur justes : 422, refus de démarrer sans modèle	2	T18, T20
	Tests verts en local et en CI	2	T14, T19
	Les deux réponses, appuyées sur des observations	3	`docs/semaine3.md`
TP4	Contrat d'entrée : cas limites, propriétés, correction dans le schéma	2	T22, T23
	Abstention : seuil mesuré et justifié, jeu d'or honnête	2,5	T24, T25
	Intégration : démarrage, modèle absent, 503, 500 ; couverture de 95 % sans exclusion abusive	2	T26
	Contrat typé, ruff et mypy muets, pre-commit, CI verte	2	T27
	`docs/semaine4.md` : chaque défaut relié à son test, chiffres du binôme	1,5	T21

Les grilles : TP5 (sur 10)

TP	Critère	Pts	Vérification
TP5	La page fonctionne : un avis, verdict, confiance, abstention	3	T29 à T31
	Les erreurs traduites pour l'utilisateur : 422, 503, API injoignable	2	T32
	Le lot d'avis : tableau, résumé, téléchargement	2	T33
	Tests du client et de la page verts, en local et en CI	1,5	T28, T34
	`docs/semaine5.md` : les trois réponses, appuyées sur des observations	1,5	Lecture

Les grilles : TP6 (sur 10)

TP	Critère	Pts	Vérification
TP6	Le paquet : `predict()` dans `sentiment_app`, l'API sans `sys.path`, le modèle configurable	2,5	T36, T37
	La commande `sentiment` : version, predict, serve, ui	2	T38
	La roue : construite sans tests ni données ni modèle, installée et essayée hors du dépôt	2,5	T39, T40
	Tests et CI verts, étape de la roue comprise	1,5	T35, T41
	`CHANGELOG.md`, étiquette `v0.6.0`, `docs/semaine6.md`	1,5	T41

Points de correction : le déplacement doit passer par `git mv` (`git log --follow` remonte alors au TP2) ; la réponse 3 du compte rendu doit citer la version de scikit-learn ou l'erreur observée.

Les signaux d'alerte

- Un seul commit géant pour tout un TP, ou un membre absent de l'historique d'un TP.
- Des fiches ou des comptes rendus identiques à ceux d'un autre binôme.
- Les valeurs du corrigé recopiées à la place des mesures du binôme.
- Un test fourni modifié pour passer ; des `# noqa` ou `# pragma: no cover` en série.
- Un jeu d'or sans cas difficile, ou dont les erreurs ont été retirées au lieu d'être déclarées.
- Des captures manquantes ou incohérentes dans `docs/journal-tp1-tp6.md`.

Les messages d'erreur rencontrés

Python et environnement

Message ou symptôme	Cause	Correction
`'pytest' n'est pas reconnu`, `No module named pytest`	Environnement non activé	`.venv\Scripts\Activate.ps1`
`Activate.ps1` : exécution de scripts désactivée	Politique d'exécution	`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`
`'deactivate' n'est pas reconnu`	Invite cmd, aucun environnement actif	Sans conséquence ; cmd : `activate.bat`
`Compiler cl cannot compile programs` (pandas)	Python plus récent que 3.11	`py -3.11 -m venv .venv`
`ResolutionImpossible` : `pyarrow==21.0.*` contre `mlflow 2.17`	Ligne `pyarrow` du kit TP1	Retirer la ligne
`No module named 'joblib'`	Dépendances jamais installées dans ce clone	`pip install -r requirements.txt`
`Could not open requirements file`	Kit TP4 non installé, ou `pip` lancé dans le kit	Installer le kit ; `pip` dans le dépôt
`DeprecationWarning` (starlette, anyio, joblib)	Versions des bibliothèques	Sans conséquence

Message ou symptôme	Cause	Correction
`1 auteur(s) de commits, 2 attendus`	B n'a pas encore commité, ou même `user.name`	Attendu après le premier push ; commit de B
`remote origin already exists`	Adresse déjà enregistrée	`git remote set-url origin <adresse>`
`Permission ... denied` to un autre compte (403)	Identifiants d'un autre compte	Compte dans l'adresse ; `cmdkey /delete:git:https://github.com`
`src refspec main does not match any`	Aucun commit, ou branche `master`	`git commit` ; `git branch -M main`
`Updates were rejected`	Dépôt créé avec un README	Le recréer vide
`The requested URL returned error: 400`	Chevrons `<depot>` laissés dans l'adresse	La vraie adresse
`Repository not found`	Invitation non acceptée, ou mauvais compte	Accepter ; compte dans l'adresse
`CRLF will be replaced by LF`	Fins de ligne Windows	Sans conséquence

Chemins, archives, et échecs attendus

Message ou symptôme	Cause	Correction
`ne ressemble pas au dépôt du TP1`	Dossier ajouté par l'Explorateur	Chemin complet du dépôt
`Expand-Archive` : chemin inexistant	Archive ailleurs ou renommée	`Get-ChildItem -Recurse -Filter`
`Rename-Item` : l'élément n'existe pas	Modèle jamais entraîné, ou pas mis de côté	`Get-ChildItem models` ; `train.py`
`unrecognized arguments: puis git status`	Deux commandes copiées comme une	Une commande par ligne
`45 passed` au lieu de 48	Autre chaîne de kits (TP3 interface)	Les kits de ce guide
Fiches : `rubriques vides`, `cocher exactement une`	Attendu (T1) : fiches vides	TP1, étape 3
`approche`		
`jeu d'or incomplet : 0 avis sur 20`	Attendu (T21)	Les vingt avis
ruff : B904, I001, F401, B905, E741	Attendu : départ du TP4	Étape E
`2 failed`, `Falsifying example`	Attendu : le défaut de l'étape B	`field_validator`

API et CI

Message ou symptôme	Cause	Correction
`FileNotFoundError: models/sentiment.joblib absent`	T20 s'il a été mis de côté ; sinon jamais entraîné	Remettre le modèle, ou `train.py`
`[WinError 10048]` : adresse déjà utilisée	Une autre API tourne	Fermer, ou `--port 8001`
CI : `No module named 'api'`	`pytest.ini` sans `pythonpath = .`	Ajouté par le kit du TP3
CI verte, tests de l'API ignorés	La CI n'entraîne pas le modèle	Étape d'entraînement avant les tests
CI du TP4 : outils introuvables	Étapes sans activation de l'environnement	Les quatre étapes de `ci-tp4.yml`

Interface Streamlit

Message ou symptôme	Cause	Correction
Streamlit demande une adresse électronique	Accueil du premier lancement	Entrée, ou `--server.headless true`
« API injoignable » en haut de la page	L'API n'est pas lancée	`uvicorn api.main:app` dans un autre terminal
Port 8501 déjà utilisé	Une autre page Streamlit tourne	La fermer, ou `--server.port 8502`
`streamlit 1.39.1 requires packaging<25`	Les outils du TP4 installent un `packaging` plus récent	Sans conséquence : les tests passent
`test_lot_et_resume` échoue en CI, pas sur le poste	La CI installe pytest 9, qui ne restaure plus `app.state`	Épingler `pytest==8.3.*` ; kit du TP5 republié

Paquet (TP6)

Message ou symptôme	Cause	Correction
`No module named 'predict'`	Le fichier de compatibilité `src/predict.py` manque	Le recréer depuis `extraits/`
`sentiment` n'est pas reconnu	Paquet non installé, ou environnement inactif	`pip install -e .`, environnement activé
`modèle introuvable : ...\site-packages\models\ ...`	`SENTIMENT_MODELE` absente, hors du dépôt	Définir la variable
`AttributeError: ... no attribute 'multi_class'`	Modèle et roue de versions de scikit-learn différentes	Entraîner et relire en 1.5
Une ancienne version installée	Une vieille roue restée dans `dist/`	Vider `dist/` avant `build`
L'étiquette absente de GitHub	`git push` n'envoie pas les étiquettes	`git push origin v0.6.0`
`OptimizeWarning: Unknown solver options: iprint`	Version récente de SciPy	Sans conséquence

Annexes

Récapitulatif des contrôles : TP1

Contrôle	Commande ou essai	Résultat attendu
T0	`Get-ChildItem -Force`	Sept dossiers, `.github`, `.gitignore`
T1	`python -m pytest -q`	3 failed, 31 passed, 4 skipped
T2	`python -m pytest tests/test_cadrage.py -q`	12 passed
T3	`git status` avant le commit	Ni `.venv/`, ni `.env`, ni `.joblib`
T4	`python scripts/check_setup.py` (membre B)	0 erreur(s)
T5	`git shortlog -sn --all` ; `test_git_history.py`	Deux noms ; 3 passed
T6	`check_setup.py` ; `python -m pytest -q`	0 erreur, 0 avertissement ; 38 passed ; CI verte
T7	La suite sur un clone frais	38 passed

Récapitulatif des contrôles : TP2

Contrôle	Commande ou essai	Résultat attendu
C1	`python -m pytest -q`	2 failed, 38 passed, 2 skipped
C2	`python src/make_demo.py`	290 avis, 145 de chaque classe
C3	`python src/train.py` ; `git status --short`	F1 0,879 ; `metrics.json` seul
C4	`python src/evaluate.py`	7 erreurs sur 58
C5	`train.py --allocine` ; `python -m pytest -q`	F1 0,920 ; 42 passed ; CI verte
C6	`python src/expliquer.py`	Sortie $-0{,}1343$ retrouvée ; 4 passed

Récapitulatif des contrôles : TP3

Contrôle	Commande ou essai	Résultat attendu
T14	`python -m pytest -q`	48 passed
T15 à T17	`/health`, avis positif, avis négatif	200 OK ; `POSITIVE` 0,777 ; `NEGATIVE` 0,7465
T18	Texte vide, champ absent, nombre	422 trois fois
T19	`git push`, onglet Actions	Entraînement, puis 48 passed
T20	`uvicorn api.main:app` sans modèle	`FileNotFoundError`

Récapitulatif des contrôles : TP4

Contrôle	Commande ou essai	Résultat attendu
T21	`python -m pytest -q`	1 failed, 51 passed, 1 skipped
T22	`/docs`, trois espaces	422, `detail` en liste
T23	`test_limites.py`, trois fois	10 passed
T24	`/docs`, mots inventés	`INDETERMINE`
T25	`test_jeu_or.py`	Aucun failed, erreurs connues en xfailed
T26	`pytest --cov=api --cov-fail-under=95`	Pannes en 500 ; 95 % atteint
T27	`ruff`, `mypy`, `pytest`, push	Muets ; aucun failed ; CI verte
T27a	`python src/biais.py`	Retournement 0,588 contre 1,000 ; 8 passed

Récapitulatif des contrôles : TP5

Contrôle	Commande ou essai	Résultat attendu
T28	`python -m pytest -q`	Aucun failed, douze tests de plus
T29	`streamlit run ui/app.py`	« API prête »
T30	Un avis positif, un avis négatif	Avis positif 78 %, avis négatif 75 %
T31	Des mots inventés	« Je ne sais pas », 52 %
T32	Trois espaces ; l'API arrêtée	« Texte refusé » ; « API injoignable »
T33	`data/avis_exemple.csv`	3 positifs, 2 négatifs, 1 à vérifier ; téléchargement
T34	`git push`	CI verte
T34a	Onglet fichier, `avis_exemple.csv`	3, 2, 1, 0 ; confiances 1, 1, 3, 1, 0 ; 9 passed

Récapitulatif des contrôles : TP6

Contrôle	Commande ou essai	Résultat attendu
T35	`python -m pytest -q`	2 failed, voulus
T36	`python -m pytest -q` ; `python src/predict.py "..."`	1 failed ; `positive (confiance 78%)`
T37	pytest, `Select-String`, ruff, mypy	Aucun failed ; plus de `sys.path` dans `api/`
T38	`sentiment --version`, `predict`, `serve`	`0.6.0` ; `POSITIVE` 0.777 ; `INDETERMINE` 0.5162 ; l'API répond
T39	`python -m build --wheel`	Une roue `0.6.0` : `api`, `sentiment_app`, `ui`
T40	La roue dans un environnement neuf	« modèle introuvable » sans la variable ; `POSITIVE` avec
T41	`git push`, `git push origin v0.6.0`	CI verte, étape de la roue comprise ; étiquette visible
T41a	`python -m pytest tests/test_fiche_modele.py -q`	13 failed voulus, puis 14 passed

Annexe A · test_cadrage.py : les constantes et les deux outils

```
REQUIRED_SECTIONS = ["Besoin en une phrase", "Utilisateur final", "Approche retenue", "Justification",
                     "Données nécessaires", "Métrique de succès", "Conséquence d'une erreur", "Risques éthiques"]
CRITERIA = ["(a)", "(b)", "(c)", "(d)"]

def fiches(cadrage_dir: Path):
    return sorted(cadrage_dir.glob("cas*.md"))

def section_body(text: str, title: str) -> str:
    pattern = re.compile(rf"^## {re.escape(title)}.*?$\n(.*?)(?=^## |\Z)", re.M | re.S)
    m = pattern.search(text)
    return m.group(1).strip() if m else ""
```

- Les huit rubriques attendues, et les quatre critères : (a) données étiquetées, (b) vérifiabilité, (c) coût par requête, (d) conséquence d'une erreur.
- `fiches()` ne retient que les fichiers `cas*.md`, triés : `EXEMPLE-corrige.md` n'est jamais contrôlé, et les erreurs apparaissent toujours d'abord sur `cas1`.
- `section_body()` renvoie le texte d'une rubrique, entre son titre et le titre suivant. La fixture `cadrage_dir`, fournie par `tests/conftest.py`, désigne `docs/cadrage/`.

Annexe A · l'expression régulière de section_body()

Morceau	Ce qu'il reconnaît
`^##`	Une ligne qui commence par `##` , un titre de niveau 2
`{re.escape(title)}`	Le titre exact ; `re.escape` neutralise les caractères spéciaux, comme l'apostrophe
`.*?$`	La fin de la ligne de titre, quelle qu'elle soit
`(.*?)`	Ce qu'on capture : le corps de la rubrique, le plus court possible
`(?=^## |\Z)`	La limite : avant le prochain `##` en début de ligne, ou la fin du fichier

`re.M` fait agir `^` et `$` à chaque ligne ; `re.S` laisse le point traverser les sauts de ligne. Pour cette fiche, `section_body(texte, "Justification")` renvoie les deux lignes entre les deux titres :

```
## Justification
> Citez au moins deux critères.
(a) Aucune donnée étiquetée n'existe. (d) Une erreur peut être grave.

## Données nécessaires
```

Annexe A · les cinq tests de test_cadrage.py

Test	Nb	Ce qu'il vérifie	Échoue si
`test_three_fiches_exist`	1	Au moins trois fichiers `cas*.md`	Une fiche supprimée ou renommée
`test_sections_present_in_every_fiche`	8	Grâce à `parametrize`, un test par rubrique : son titre figure dans chaque fiche	Un titre modifié : `## justification`, `### Justification`
`test_sections_are_filled`	1	Au moins 20 caractères rédigés par rubrique, hors lignes vides, consignes (`>`) et cases (`- []`)	Une rubrique trop courte ; le message les liste toutes
`test_exactly_one_approach_checked`	1	Exactement une fois `- [x]` dans la fiche	Aucune ou deux cases ; `- [X]`, `-[x]` ne comptent pas
`test_justification_cites_two_criteria`	1	Au moins deux de `(a)` à `(d)` sous « Justification »	Critères cités ailleurs, ou écrits `a)`, `(A)`

Les trois derniers tests s'arrêtent à la première fiche fautive ; `test_sections_are_filled` rassemble toutes les rubriques trop courtes. Les tests vérifient la présence des lettres : que les critères s'appliquent vraiment au cas, c'est l'enseignant qui le juge, pour 4 points sur 10.

Annexe B · les sept erreurs du jeu de démonstration

Vérité	Prédit	Avis	Retournement
positive	negative	Taille trop petite et retour payant au déballage, sauf que ensuite produit conforme à la description, très satisfait.	sauf que ensuite
positive	negative	Commande jamais reçue et remboursement en attente au début, sauf que ensuite je recommande sans hésiter.	sauf que ensuite
negative	positive	Service impeccable et livraison en avance sur le papier, sauf que ensuite produit cassé à l'arrivée.	sauf que ensuite
positive	negative	Facture erronée, aucun geste commercial sur le papier, puis rapidement service impeccable et livraison en avance.	puis rapidement
positive	negative	Commande jamais reçue et remboursement en attente au début, avant que emballage soigné et finitions propres.	avant que
negative	positive	Service impeccable et livraison en avance les premiers jours, avant que notice incompréhensible et pièces manquantes.	avant que
positive	negative	Se casse au bout de deux utilisations à la livraison, mais finalement excellent rapport qualité prix.	mais finalement

Ce sont tous des avis mitigés. Dans les sept cas, la prédiction suit la première partie de l'avis, alors que le verdict est donné par la fin. `python src/evaluate.py` n'en affiche que trois.

Annexe B · le compte rendu de la semaine 2

Ce qu'on attend, en trois phrases

1. **Quelles erreurs** : les 7, toutes sur des avis mitigés, avec un exemple cité.
2. **Pourquoi** : le modèle additionne les poids des mots sans tenir compte de leur ordre ; un sac de mots ne voit pas qu'un connecteur comme « sauf que » renverse le sens de ce qui précède.
3. **Que faire** : une représentation qui tient compte de l'ordre et du contexte, comme les embeddings de phrase de la semaine 9.

Trois remarques

Vérifier ses chiffres (matrice 27 2 / 5 24, sinon relancer `python src/train.py`). Ne pas proposer les bigrammes : le vectoriseur en utilise déjà, comme « service client ». Écrire avec ses propres mots : des comptes rendus identiques sont un signal d'alerte. Ce critère vaut 2 points sur 10.

Annexe C · train.py : les constantes et les données

Constante	Rôle
`RACINE`	La racine du dépôt, calculée depuis le script : il fonctionne quel que soit le dossier courant
`DEMO`	`data/demo_reviews.csv`, écrit par `make_demo.py`
`URL_ALLOCINE`	Le jeu public Allociné, au format Parquet sur Hugging Face : 20 000 critiques
`MODELE, METRIQUES`	Les deux fichiers produits : `models/sentiment.joblib` et `models/metrics.json`
`SEUIL_F1`	0,85 : le script l'affiche, c'est `test_model.py` qui le fait respecter
`GRAINE`	42 : deux exécutions donnent le même découpage et le même modèle

`charger(allocine)` renvoie un tableau à deux colonnes, `review` et `label`, et le nom de la source. Pour Allociné, il télécharge le fichier (connexion et pyarrow requises) et convertit 0 et 1 en `negative` et `positive`. Sans `demo_reviews.csv`, il s'arrête en disant de lancer `make_demo.py`. `evaluate.py` et `courbe_seuil.py` importent `GRAINE`, `MODELE` et `charger` : ils retrouvent le même jeu de test.

Annexe C · train.py : le pipeline

```
Pipeline([
    ("tfidf", TfidfVectorizer(ngram_range=(1, 2), min_df=2, sublinear_tf=True)),
    ("clf", LogisticRegression(max_iter=1000, class_weight="balanced", random_state=GRAINE)),
])
```

Réglage	Effet
`ngram_range=(1, 2)`	Les mots seuls et les paires de mots consécutifs, comme « service client »
`min_df=2`	Un terme présent dans moins de deux avis est ignoré : 316 termes sur le jeu de démonstration
`sublinear_tf=True`	Un mot répété compte de moins en moins : $1 + \log(\text{fréquence})$
`max_iter=1000`, `class_weight="balanced"`	Le temps de converger ; compenser un déséquilibre des classes

Le vectoriseur et le classifieur forment un seul objet, sauvegardé d'un bloc : en production, `predict()` applique forcément la même transformation qu'à l'entraînement.

Annexe C · train.py : main(), étape par étape

1. **Découper avant toute transformation** : `train_test_split(..., test_size=0.2, random_state=GRAINE, stratify=...)`. 232 avis d'entraînement et 58 de test ; 16 000 et 4 000 sur Allociné. Sinon, le vocabulaire du jeu de test fuirait dans l'entraînement.
2. **Entraîner** : `pipeline.fit(X_train, y_train)`, sur les seuls avis d'entraînement.
3. **Mesurer** sur les 58 avis jamais vus : le F1 macro, moyenne des F1 des deux classes.
4. **Sauvegarder** : `joblib.dump` écrit le modèle, hors de Git ; `metrics.json` reçoit source, date, tailles, F1, exactitude, rappel des négatifs, seuil, et il est versionné.

```
Source : demo, 290 avis (145 négatifs)
232 avis d'entraînement, 58 de test
Vocabulaire : 316 termes
F1 macro sur le jeu de test : 0.879 (seuil 0.85)
Modèle : models/sentiment.joblib
Métriques : models/metrics.json (versionné dans Git, contrairement au modèle)
```

`--allocine` écrase le modèle de démonstration ; `metrics.json` change à chaque entraînement, à cause de la date.

Annexe D · le lancement de l'API et le chemin d'une requête

```
INFO     Démarrage de l'API...
INFO     Modèle chargé avec succès : tfidf-logreg (TP2, models/sentiment.joblib)
INFO:    Application startup complete.
```

`uvicorn` lance l'application `app` de `api/main.py`. La fonction `lifespan` charge le modèle une seule fois, avant la première requête, dans `app.state.classifier`. Ensuite, chaque requête suit le même chemin :

1. FastAPI choisit la fonction de la route : `health_check` ou `predict` ;
2. Pydantic vérifie le corps avec `PredictRequest` ; sinon 422, sans exécuter la fonction ;
3. `_classifier()` récupère le modèle, ou répond 503 s'il manque ;
4. `api/model.py` appelle `predict()` du TP2 et met l'étiquette en majuscules ;
5. la réponse est vérifiée par `PredictResponse`, puis renvoyée en JSON.

Annexe D · T15 à T17 : les réponses attendues

```
GET /health   {"status": "OK", "model": "tfidf-logreg (TP2, models/sentiment.joblib)"}
POST /predict {"label": "POSITIVE", "score": 0.777, "text": "Livraison rapide, produit conforme, très satisfait"}
POST /predict {"label": "NEGATIVE", "score": 0.7465, "text": "Produit cassé et service injoignable, à fuir"}
```

Champ	Ce qu'il contient
`label`	L'étiquette du modèle, mise en majuscules par `api/model.py`
`score`	La confiance dans la classe prédite : la plus grande des deux probabilités, entre 0,5 et 1
`text`	Les 100 premiers caractères de l'avis, pour vérifier ce qui a été analysé

`/health` sert aux outils de déploiement et à la page du TP5. Les scores sont exacts parce que le modèle de démonstration est entraîné avec une graine fixée ; d'autres valeurs signalent le modèle Allociné.

Annexe D · T18 : trois refus 422

Corps envoyé	type	loc	msg
`{"text": ""}`	`string_too_short`	`["body", "text"]`	String should have at least 1 character
`{}`	`missing`	`["body", "text"]`	Field required
`{"text": 42}`	`string_type`	`["body", "text"]`	Input should be a valid string

Pydantic refuse ces corps avant que la fonction ne s'exécute ; `detail` est une liste d'erreurs. C'est un 422 et non un 500 : l'erreur vient du client, le serveur fonctionne.

À observer, sans le corriger : trois espaces

`{"text": "   "}` passe le contrat, puis `predict()` le refuse : `{"detail": "texte vide"}`, une simple chaîne. Deux formes pour le même refus : le premier défaut que le TP4 fera trouver.

T19 : la CI entraîne, puis teste

Le kit du TP3 fait installer FastAPI, httpx et uvicorn par la CI, et ajoute l'étape « Modèle de démonstration » avant les tests : sans elle, les six tests de l'API seraient ignorés, faute de modèle. `pythonpath = .` permet aux tests de trouver `api`. Attendu : `48 passed`.

T20 : sans modèle, l'API refuse de démarrer. `lifespan` appelle `load_model()`, puis `charger_modele()`, qui lève l'erreur :

```
ERROR:    Traceback (most recent call last):
FileNotFoundError: models/sentiment.joblib absent : lancez python src/train.py
ERROR:    Application startup failed. Exiting.
```

Une API sans modèle ne répondra jamais correctement : en refusant de démarrer, elle rend la panne visible au déploiement. Le 503 sert aux indisponibilités passagères.

Annexe E · la page /docs : l'en-tête et les routes

La page `http://localhost:8000/docs` est construite à partir du code : décorateurs et docstrings de `api/main.py`, contrats de `api/schemas.py`. Modifier le code modifie la page.

- **L'en-tête** : « Sentiment Analysis API », la description, l'étiquette `1.0.0` (la version de l'API), l'étiquette `OAS 3.1` (la norme OpenAPI) et le lien `/openapi.json`.
- **Les routes**, regroupées sous « default » :

Barre	Route	Résumé	Fonction
GET, en bleu	`/`	Accueil	`root`
GET, en bleu	`/health`	Santé de l'API	`health_check`
POST, en vert	`/predict`	Classifier un texte	`predict`

Annexe E · une route dépliée, les schémas, les pages voisines

- **Request body** : l'exemple de `schemas.py` et le contrat, une chaîne de 1 à 2000 caractères.
- **Responses** : 200 et 422 sont documentés. 503 et 500 existent dans le code, mais la page ne les montre pas : une limite de la documentation automatique.
- **« Try it out », puis « Execute »** : la commande Curl équivalente, l'adresse appelée, puis le code et le JSON reçus. Sous Windows, préférez les commandes `httpx` à la commande Curl.
- **« Schemas »** : `PredictRequest`, `PredictResponse`, `HealthResponse`, et `HTTPValidationError`, `ValidationError` ajoutés par FastAPI. `ErrorResponse`, inutilisé, n'apparaît pas.
- **Les pages voisines** : `/redoc`, la même documentation en lecture seule ; `/openapi.json`, sa version en JSON ; `/`, l'accueil de l'API.

Annexe E · vingt essais (1 à 10)

N°	Essai	Attendu	Ce que cela montre
1	`GET /`	200, liens vers `/docs` et `/health`	L'accueil
2	`GET /health`	200, `"status": "OK"`	L'API est prête (T15)
3	Avis positif évident	200, `POSITIVE`, 0,777	T16
4	Avis négatif évident	200, `NEGATIVE`, 0,7465	T17
5	`{"text": ""}`	422, `string_too_short`	Le texte vide est refusé (T18)
6	`{}`	422, `missing`	Le champ est obligatoire (T18)
7	`{"text": 42}`	422, `string_type`	Une chaîne est attendue (T18)
8	`{"text": null}`	422, `string_type`	`null` n'est pas une chaîne
9	`{"text": "a"}`	200, `NEGATIVE`, 0,5162	Un caractère suffit au contrat
10	2000 caractères	200, `NEGATIVE`, 0,5162	La limite haute est acceptée

Pour préparer 2000 caractères : `python -c "print('a' * 2000, end='')" | Set-Clipboard`, puis coller entre les guillemets.

Annexe E · vingt essais (11 à 20)

N°	Essai	Attendu	Ce que cela montre
11	2001 caractères	422, `string_too_long`	Au-delà, le contrat refuse
12	`{"text": "  "}`	422, `{"detail": "texte vide"}` : une chaîne	Deux formes de refus : corrigé au TP4
13	Mots inventés	200, `NEGATIVE`, 0,5162	Le modèle devine ; abstention au TP4
14	L'avis mitigé « Taille trop petite…, sauf que ensuite… »	200, `NEGATIVE`, 0,5335, au lieu de positif	La limite du sac de mots
15	`{"text": "Très satisfait", "langue": "fr"}`	200, `POSITIVE`, 0,7461	Un champ en trop est ignoré
16	Un corps qui n'est pas du JSON, avec `httpx`	422, `json_invalid`	Le corps doit être du JSON
17	`http://localhost:8000/predict`, dans le navigateur	405, Method Not Allowed	`/predict` n'accepte que POST
18	`http://localhost:8000/inconnue`	404, Not Found	Route inexistante
19	`http://localhost:8000/redoc`	La documentation, autre présentation	La même description OpenAPI
20	L'API lancée sans modèle	Pas de page : l'API ne démarre pas	T20

0,5162 revient trois fois (essais 9, 10, 13) : sans mot connu, la représentation TF-IDF est nulle, et il ne reste que l'ordonnée à l'origine de la régression logistique.

PowerShell, cmd et bash

Action	PowerShell	cmd	bash
Créer l'environnement	`py -3.11 -m venv .venv`	`py -3.11 -m venv .venv`	`python3.11 -m venv .venv`
L'activer	`.venv\Scripts\Activate.ps1`	`.venv\Scripts\activate.bat`	`source .venv/bin/activate`
Le supprimer	`Remove-Item -Recurse -Force .venv`	`rmdir /s /q .venv`	`rm -rf .venv`
Renommer	`Rename-Item a b`	`ren a b`	`mv a b`
Lister, fichiers cachés	`Get-ChildItem -Force`	`dir /a`	`ls -a`
Afficher un fichier	`Get-Content f`	`type f`	`cat f`
Dossier temporaire	`cd $env:TEMP`	`cd %TEMP%`	`cd /tmp`
Variable d'environnement	`$env:API_URL = "..."`	`set API_URL=...`	`export API_URL=...`

`python`, `pip`, `git`, `uvicorn`, `ruff`, `mypy`, `pre-commit` s'écrivent de la même façon partout.

Une commande par ligne,
un contrôle vert avant de continuer.